

MAS Design Thinking Workshop

Pen, Paper & Brain | No Code Required

Design your Multi-Agent System before you write a single line of code.

8 Design Questions Every Developer Must Answer

- 1 What problem do you want to solve?
- 2 Why MAS and not a single agent?
- 3 How many agents do you need?
- 4 What does each agent do?
- 5 How will agents communicate?
- 6 Cooperative, competitive, or both?
- 7 Fully observable or partially observable environment?
- 8 How does MAS fit your frontend / backend?

Platforms to implement your design:

Google ADK

A2A Protocol

CrewAI

LangGraph

AutoGen AG2

Claude API

Workshop Overview

Before developers write a single line of code, they need to think clearly about system design. This workshop takes you back to basics: a pen, paper, and your brain. For each of the 10 business scenarios, you will work through 8 design questions that experienced MAS architects ask before touching a keyboard.

How to Use This Workbook

Step 1: Read the scenario	Each scenario gives you a real business problem with context, constraints, and success metrics.
Step 2: Answer Q1-Q4 (Page A)	Define the problem, justify MAS, enumerate agents, and detail each agent's role.
Step 3: Answer Q5-Q8 (Page B)	Design the communication protocol, collaboration mode, environment type, and system integration.
Step 4: Draw the diagram	Sketch agent nodes, message flows, and orchestration hierarchy on the dot-grid canvas.
Step 5: Compare & discuss	Share your design with the group. Different architectures solve the same problem differently.
Step 6: Implement (Code Lab)	Use the exercises PDF to implement your chosen scenario in CrewAI, LangGraph, or ADK.

Learning Objectives

- Identify when a problem genuinely requires multiple agents vs. a single agent
- Enumerate agents by responsibility and define their goals, tools, and state
- Design inter-agent communication using FIPA-ACL performatives and domain ontologies
- Choose the right collaboration model (cooperative, competitive, or mixed)
- Classify the environment observability and determine state estimation needs
- Map your MAS design to a concrete implementation pattern in Google ADK or CrewAI
- Connect agent outputs to a real frontend/backend application architecture

Important Design Principle

Remember: "More agents = more coordination overhead. Start with the fewest agents possible. Each new agent must justify its existence."

Difficulty Scale

Level	Dots	Agent Count	Communication Complexity	Scenarios
Beginner	**	2-3 agents	Sequential, simple messages	1, 2, 7
Intermediate	***	4-5 agents	Parallel flows, structured ontology	3, 4, 9, 10
Advanced	****	5+ agents	Contract Net, event streams, A2A	5, 6, 8

The 8 Design Questions

Answer these before writing any code. Keep this page open during the workshop.

Q1 What problem do you want to solve?

- Write a crisp 2-3 sentence problem statement.
- Include: Who is affected? What is failing? What is the cost/impact?
- Define measurable success criteria (KPIs).

Example: "A 300-room hotel wastes 40% energy due to no coordination between HVAC zones. Goal: reduce energy cost by 30% without degrading guest comfort score below 4.5/5."

Q2 Why do you need MAS and not a single agent?

- Is the problem too large for one context window?
- Does it have independent parallel workstreams that benefit from concurrency?
- Does it require specialized expertise in distinct domains?
- Does it need fault tolerance (one agent fails, others continue)?

If "No" to all of the above, a single agent may be sufficient. Challenge yourself!

Q3 How many agents do you need?

- Rule of thumb: one agent per distinct role or responsibility boundary.
- Name each agent and assign it exactly ONE primary goal.
- Ask: could two agents be merged without harming separation of concerns?
- More agents = more coordination overhead. Justify each one.

Example: OccupancyAgent (sense), ComfortAgent (model preference), HVACAgent (control), BudgetAgent (enforce limits).

Q4 What does each agent do? What are its functions and tools?

- For each agent: Goal, Backstory/Persona, Tools it can call, Output it produces.
- LLM Agent: reasons, decides, generates text. Code Agent: executes deterministic logic.
- What data does it read? What state does it write?
- What happens if this agent fails?

HVACAgent: Goal="Maintain target temperature per zone", Tools=[read_sensor(), set_temp(), read_schedule()], Output=control_command.

The 8 Design Questions (continued)

Q5 How will agents communicate? Language, ontology, protocol?

- Language: natural language (LLM-to-LLM) or structured schema (JSON/Pydantic)?
- Ontology: define the shared vocabulary. What objects, actions, attributes exist in this domain?
- FIPA-ACL performatives: REQUEST, INFORM, QUERY-IF, PROPOSE, CFP, AGREE, FAILURE...
- Protocol: who initiates? What are valid message sequences? (Request-Reply, Contract Net, Blackboard...)
- Modern: Google A2A for cross-framework; ADK shared session.state for same-framework.

BudgetAgent sends CFP(task=reduce_load, zone=lobby, by=15%). HVACAgent responds PROPOSE(action=raise_setpoint, saving=12%). BudgetAgent replies ACCEPT-PROPOSAL.

Q6 How do agents collaborate? Cooperative, competitive, or both?

- Cooperative: agents share a common goal and divide labor. (Most MAS are cooperative.)
- Competitive: agents compete for tasks or resources. Market-based / auction systems.
- Mixed: some agents cooperate within a team while teams compete for contracts.
- Adversarial: some agents actively try to detect or counter others (security, fraud).

In a supply chain: SupplierAgents compete via auction (Contract Net). Once selected, WarehouseAgent and DeliveryAgent cooperate to fulfill the order.

Q7 What is the agents' environment? Fully or partially observable?

- Fully observable: every agent can see all relevant state at all times. Simpler coordination.
- Partially observable: agents see only their local state. Need state estimation, belief updates.
- Static vs. dynamic: does the environment change while agents are deliberating?
- Do agents need memory (episodic, semantic, working)?
- Need state estimation? Use Bayesian filtering, Kalman filter, or LLM-based inference.

Hotel energy system: each zone agent sees only its own sensors (partial). A central BudgetAgent sees all zones (fuller view). Need occupancy prediction (state estimation).

Q8 How does MAS fit your frontend and backend architecture?

- What API surface does the MAS expose? REST? WebSocket? Event stream?
- How does a human trigger agent workflows? Dashboard? Chat? Webhook?
- How do agents report results back to the frontend?
- Where does the MAS run? Cloud function? Container? Edge device?
- How do you monitor agent health, latency, and cost?

Frontend (React dashboard) -> POST /api/optimize -> MAS orchestrator -> agents run in parallel -> results streamed back via WebSocket -> dashboard updates in real time.

Agent Communication Language Reference

Keep this page handy when answering Q5 on each scenario worksheet.

FIPA-ACL Performatives

FIPA-ACL (Foundation for Intelligent Physical Agents - Agent Communication Language) defines standard message types called performatives. Each message has: performative, sender, receiver, content (proposition), conversation-id.

Performative	Meaning	Direction	Typical Use
REQUEST	Ask agent to perform action	A -> B	Trigger a task
INFORM	Assert a fact is true	A -> B	Report sensor reading / result
QUERY-IF	Ask if proposition is true	A -> B	Check availability / status
PROPOSE	Suggest a value or plan	B -> A	Bid in Contract Net
ACCEPT-PROPOSAL	Approve a proposal	A -> B	Award contract
REJECT-PROPOSAL	Decline a proposal	A -> B	Decline bid
CALL-FOR-PROPOSAL (CFP)	Broadcast task for bids	A -> All	Initiate Contract Net
AGREE	Will perform requested action	B -> A	Acknowledge request
REFUSE	Cannot or will not perform	B -> A	Capacity full
FAILURE	Could not complete action	B -> A	Report error
INFORM-DONE	Action completed successfully	B -> A	Confirm completion

Ontology Design

An ontology defines the shared vocabulary agents use to understand each other. Without a shared ontology, agents may use the same words to mean different things.

Component	Definition	Hotel Example	E-Commerce Example
Objects (Nouns)	Entities that exist in the domain	Room, Zone, HVAC, Sensor	Order, Product, Warehouse, Route
Actions (Verbs)	Things agents can do	set_temperature(), read_occupancy()	place_order(), ship_item()
Attributes	Properties of objects	temperature, setpoint, zone_id	quantity, status, delivery_date
Relations	How objects relate	Zone has_sensor Sensor	Order contains Product
Constraints	Rules and limits	temp >= 18 and temp <= 26	stock_level >= order_quantity

Google ADK & A2A Protocol Reference

Use these patterns when mapping your design to an implementation in Q8.

Google ADK Multi-Agent Patterns

ADK Pattern	When to Use	ADK Implementation	Your Scenario Mapping
Sequential Pipeline	Steps must run in strict order; each step uses previous output	SequentialAgent(sub_agents=[A,B,C])	Intake -> Classify -> Respond -> Log
Parallel Fan-Out	Independent tasks can run concurrently; faster overall execution	ParallelAgent(sub_agents=[A,B,C])	Parallel: market + legal + financial research
Loop / Monitor	Continuous or iterative work; repeat until condition met	LoopAgent(max_iterations=N)	Continuous sensor monitoring, retry until quality OK
Coordinator / Dispatcher	Dynamic routing based on intent; LLM decides which sub-agent to call	LlmAgent + transfer_to_agent()	Customer support triage dispatcher
Sub-agent as Tool	Parent LLM explicitly invokes a child agent as a callable tool	AgentTool(agent=child_agent)	Orchestrator calls SpecialistAgent as needed
Hierarchical Decomposition	Multi-level tree breaks down complex goals into steps	Nested SequentialAgent trees	City traffic: region -> district -> intersection
Generator-Critic	One agent generates, another reviews; improves output quality	SequentialAgent([Generator, Critic])	Report writer + fact checker
Human-in-the-Loop	Require human approval before high-stakes actions	FunctionTool(external_approval)	Financial transaction over threshold

Google A2A Protocol

Agent2Agent (A2A) is an open protocol enabling communication between agents built on different frameworks (LangGraph, CrewAI, ADK, custom). Agents interact without sharing internal memory, tools, or proprietary logic.

A2A Concept	Description	vs. MCP
Agent Card	JSON descriptor: agent name, capabilities, endpoint URL, auth method. Agents discover each other via Agent Cards.	MCP has Tool Catalog; A2A has Agent Card
Task	Unit of work sent from one agent to another. Has ID, state (submitted/working/done/failed), and artifacts.	MCP has Tool Call; A2A has Task lifecycle
Artifact	Data produced by an agent: file, JSON object, text, image. Returned to the calling agent.	MCP returns ToolResult; A2A returns Artifact
Multi-turn	Agents engage in back-and-forth conversation to clarify, refine, or negotiate.	MCP is typically single-call; A2A supports conversation
Streaming	Agent sends partial/incremental results in real time, not just at completion.	Both support streaming
Cross-framework	A LangGraph agent can call a CrewAI agent as a peer. No shared codebase needed.	MCP is framework-agnostic for tools; A2A extends this to agents

Key insight: Use ADK shared session.state for agents within the same application. Use A2A when your agents span different services, frameworks, or vendor systems.

A 5-star hotel with 300 guest rooms and 12 function halls needs to reduce energy costs by 30% without degrading guest comfort. HVAC, lighting, elevators, and pool heating each run on separate standalone systems with zero coordination. Occupancy fluctuates dramatically by hour, day, and season. Manual adjustments by engineering staff are slow and inconsistent.

Success Metrics	Energy cost (THB/day) Guest comfort score (target $\geq 4.5/5$) Response lag (sec)
Key Constraints	Cannot disrupt live guests. Must respect pre-set comfort zones per room category.

Q1. What specific problem are you solving? Write a clear 2-sentence problem statement.

Include: who is affected, what is failing, what is the measurable impact.

Q2. Why do you need a Multi-Agent System rather than a single agent?

Check all that apply:

- ☐ Problem too large for one context window
- ☐ Multiple independent parallel workstreams
- ☐ Requires distinct specialized expertise per domain
- ☐ Needs fault tolerance (one agent fails, others continue)
- ☐ Real-time coordination across physical systems/zones
- ☐ Different agents need different tools/permissions

Explain your reasoning:

Q3. How many agents do you need? List them below with their primary role.

Hint for this scenario: OccupancyAgent, HVACControlAgent, LightingAgent, EnergyBudgetAgent, WeatherForecastAgent

#	Agent Name	Type (LLM / Code / Both)	Primary Role (one sentence)
1			
2			
3			
4			
5			
6			

Q4. What does each agent do? Detail its functions and tools.

List agent's goal, tools it needs, what data it reads, and what it outputs.

Agent Name	Goal / Task	Tools Needed	Output / State Key

Agent Interaction Diagram

Draw agent nodes, message arrows, and hierarchy.

A2A: EnergyBudgetAgent orchestrates; zone agents are sub-agents. BMS system via A2A.

Draw your Agent Interaction Diagram here | use arrows to show message flow

Draw your Agent Interaction Diagram here | use arrows to show message flow

Notes / Design Decisions:

My Key Insight:

An online marketplace processes 50,000 orders per day from 1,000 third-party vendors spread across 20 warehouses. Inventory is fragmented, delivery routes vary by region, and flash sales cause demand spikes of 10x within minutes. Customer SLA requires 98% on-time delivery within 2 days. Returns add 15% overhead on top.

Success Metrics	On-time delivery % Fulfillment cost per order Inventory accuracy %
Key Constraints	Vendor SLAs differ. Some items require cold chain. Returns must be processed within 5 days.

Q1. What specific problem are you solving? Write a clear 2-sentence problem statement.

Include: who is affected, what is failing, what is the measurable impact.

.....

.....

.....

Q2. Why do you need a Multi-Agent System rather than a single agent?

Check all that apply:

- ☐ Problem too large for one context window
- ☐ Multiple independent parallel workstreams
- ☐ Requires distinct specialized expertise per domain
- ☐ Needs fault tolerance (one agent fails, others continue)
- ☐ Real-time coordination across physical systems/zones
- ☐ Different agents need different tools/permissions

Explain your reasoning:

.....

.....

Q3. How many agents do you need? List them below with their primary role.

Hint for this scenario: OrderIntakeAgent, InventoryAgent, WarehouseRouterAgent, DeliveryAgent, CustomerNotifyAgent

#	Agent Name	Type (LLM / Code / Both)	Primary Role (one sentence)
1			
2			
3			
4			
5			
6			

Q4. What does each agent do? Detail its functions and tools.

List agent's goal, tools it needs, what data it reads, and what it outputs.

Agent Name	Goal / Task	Tools Needed	Output / State Key

Q5. How will agents communicate?

Hint: Order events (JSON schema); status updates via INFORM; inventory queries via QUERY-IF

Message Language:

Key Ontology Terms:

Protocol / Standard:

Sample Message:

Q6. Collaboration mode?

- ☐ Cooperative — shared goal, divide labor
- ☐ Competitive — agents bid for tasks
- ☐ Mixed — cooperate within, compete across
- ☐ Adversarial — detect or counter others

Which agents compete? Which cooperate? Why?

Q7. Agent environment observability?

- ☐ Fully Observable ☐ Partially Observable
- ☐ Static (env does not change) ☐ Dynamic (env changes constantly)

What state is hidden? What needs to be estimated?

Memory type needed:

- ☐ No memory ☐ Short-term (session)
- ☐ Long-term (DB) ☐ Shared (blackboard)

Q8. Frontend / Backend integration?

How does a user trigger agents? How do results come back?

API surface (REST / WebSocket / Event stream / other):

Agent Interaction Diagram

Draw agent nodes, message arrows, and hierarchy.

Notes / Design Decisions:

My Key Insight:

ADK: SequentialAgent(validate -> allocate -> route -> dispatch -> notify)

A2A: External warehouse management and logistics APIs connected via A2A.

An urban emergency department with 80 beds handles 400+ patient arrivals per day. Triage nurses are overwhelmed during peak hours. Specialist availability changes every hour. Medication errors occur when multiple care teams update patient records concurrently. Critical patients sometimes wait too long due to poor visibility of bed status.

Success Metrics	Triage-to-treatment time (min) Bed utilization % Medication error rate
Key Constraints	Patient privacy (HIPAA/PDPA). Life-critical decisions need physician confirmation.

Q1. What specific problem are you solving? Write a clear 2-sentence problem statement.

Include: who is affected, what is failing, what is the measurable impact.

Q2. Why do you need a Multi-Agent System rather than a single agent?

Check all that apply:

- ☐ Problem too large for one context window
 ☐ Multiple independent parallel workstreams
- ☐ Requires distinct specialized expertise per domain
 ☐ Needs fault tolerance (one agent fails, others continue)
- ☐ Real-time coordination across physical systems/zones
 ☐ Different agents need different tools/permissions

Explain your reasoning:

Q3. How many agents do you need? List them below with their primary role.

Hint for this scenario: TriageAgent, DiagnosticSupportAgent, BedManagementAgent, PharmacyAgent, SpecialistCoordAgent

#	Agent Name	Type (LLM / Code / Both)	Primary Role (one sentence)
1			
2			
3			
4			
5			
6			

Q4. What does each agent do? Detail its functions and tools.

List agent's goal, tools it needs, what data it reads, and what it outputs.

Agent Name	Goal / Task	Tools Needed	Output / State Key

Q5. How will agents communicate?

Hint: HL7 FHIR messages for clinical data; FIPA REQUEST/INFORM for agent coordination

Message Language: _____

Key Ontology Terms: _____

Protocol / Standard: _____

Sample Message: _____

Q6. Collaboration mode?

- ☐ Cooperative — shared goal, divide labor
- ☐ Competitive — agents bid for tasks
- ☐ Mixed — cooperate within, compete across
- ☐ Adversarial — detect or counter others

Which agents compete? Which cooperate? Why?

Q7. Agent environment observability?

- ☐ Fully Observable ☐ Partially Observable
- ☐ Static (env does not change) ☐ Dynamic (env changes constantly)

What state is hidden? What needs to be estimated?

Memory type needed:

- ☐ No memory ☐ Short-term (session)
- ☐ Long-term (DB) ☐ Shared (blackboard)

Q8. Frontend / Backend integration?

How does a user trigger agents? How do results come back?

API surface (REST / WebSocket / Event stream / other):

Agent Interaction Diagram

Draw agent nodes, message arrows, and hierarchy.

Notes / Design Decisions:

My Key Insight:

ADK: LoopAgent for continuous vital-sign monitoring;
SequentialAgent for care pathway steps

A2A: Hospital MAS communicates with lab system, radiology, and pharmacy via A2A.

A printed circuit board factory runs 3 production lines at 10,000 units/day. Current defect rate is 2.3%, causing \$180,000 monthly losses. Root causes (machine calibration drift, bad material batches, operator error) are hard to pinpoint. Maintenance windows cannot halt production for more than 4 hours per month total.

Success Metrics	Defect rate % MTTR (mean time to repair, hrs) Production uptime %
Key Constraints	ISO 9001 traceability required. Maintenance agent cannot halt a line without supervisor approval.

Q1. What specific problem are you solving? Write a clear 2-sentence problem statement.

Include: who is affected, what is failing, what is the measurable impact.

Q2. Why do you need a Multi-Agent System rather than a single agent?

Check all that apply:

- ☐ Problem too large for one context window
 ☐ Multiple independent parallel workstreams
- ☐ Requires distinct specialized expertise per domain
 ☐ Needs fault tolerance (one agent fails, others continue)
- ☐ Real-time coordination across physical systems/zones
 ☐ Different agents need different tools/permissions

Explain your reasoning:

Q3. How many agents do you need? List them below with their primary role.

Hint for this scenario: VisionInspectionAgent, DefectClassifierAgent, RootCauseAgent, MaintenanceSchedulerAgent, ProductionAdjAgent

#	Agent Name	Type (LLM / Code / Both)	Primary Role (one sentence)
1			
2			
3			
4			
5			
6			

Q4. What does each agent do? Detail its functions and tools.

List agent's goal, tools it needs, what data it reads, and what it outputs.

Agent Name	Goal / Task	Tools Needed	Output / State Key

Q5. How will agents communicate?

Hint: Machine telemetry (OPC-UA protocol) + defect events + work orders via JSON

Message Language: _____

Key Ontology Terms: _____

Protocol / Standard: _____

Sample Message: _____

Q6. Collaboration mode?

- ☐ Cooperative — shared goal, divide labor
- ☐ Competitive — agents bid for tasks
- ☐ Mixed — cooperate within, compete across
- ☐ Adversarial — detect or counter others

Which agents compete? Which cooperate? Why?

Q7. Agent environment observability?

- ☐ Fully Observable ☐ Partially Observable
- ☐ Static (env does not change) ☐ Dynamic (env changes constantly)

What state is hidden? What needs to be estimated?

Memory type needed:

- ☐ No memory ☐ Short-term (session)
- ☐ Long-term (DB) ☐ Shared (blackboard)

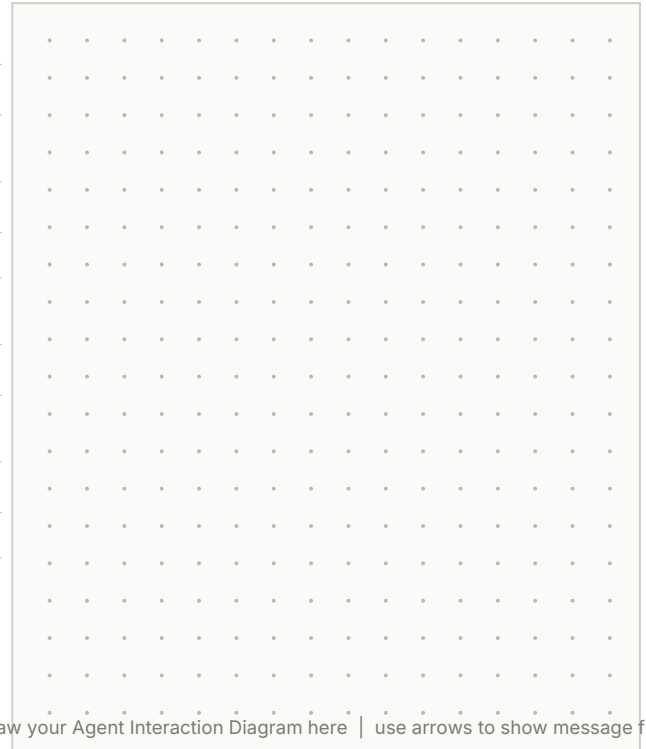
Q8. Frontend / Backend integration?

How does a user trigger agents? How do results come back?

API surface (REST / WebSocket / Event stream / other):

Agent Interaction Diagram

Draw agent nodes, message arrows, and hierarchy.



Draw your Agent Interaction Diagram here | use arrows to show message flow

Notes / Design Decisions:

My Key Insight:

ADK: ParallelAgent across 3 lines; LoopAgent for continuous monitoring; Human-in-the-loop for halt decisions

A2A: MES (Manufacturing Execution System) and ERP integration via A2A.

A digital bank processes 5 million transactions per day with fraud losses of 0.08% (\$2.4M/month). Existing rule-based systems generate 40% false positives, blocking legitimate customers. New fraud patterns (account takeover, synthetic identity, mule networks) emerge weekly. Detection and response must happen within 200ms to meet regulatory and UX requirements.

Success Metrics	Fraud loss (\$) False positive rate % Detection-to-action latency (ms)
Key Constraints	Cannot block legitimate transactions. Regulatory reporting within 24 hours of confirmed fraud.

Q1. What specific problem are you solving? Write a clear 2-sentence problem statement.

Include: who is affected, what is failing, what is the measurable impact.

Q2. Why do you need a Multi-Agent System rather than a single agent?

Check all that apply:

- ☐ Problem too large for one context window
 ☐ Multiple independent parallel workstreams
- ☐ Requires distinct specialized expertise per domain
 ☐ Needs fault tolerance (one agent fails, others continue)
- ☐ Real-time coordination across physical systems/zones
 ☐ Different agents need different tools/permissions

Explain your reasoning:

Q3. How many agents do you need? List them below with their primary role.

Hint for this scenario: TransactionMonitorAgent, AnomalyDetectorAgent, RiskScorerAgent, AccountActionAgent, CustomerNotifyAgent

#	Agent Name	Type (LLM / Code / Both)	Primary Role (one sentence)
1			
2			
3			
4			
5			
6			

Q4. What does each agent do? Detail its functions and tools.

List agent's goal, tools it needs, what data it reads, and what it outputs.

Agent Name	Goal / Task	Tools Needed	Output / State Key

Q5. How will agents communicate?

Hint: Real-time event stream (Kafka topics); FIPA CFP for investigation task assignment

Message Language: _____

Key Ontology Terms: _____

Protocol / Standard: _____

Sample Message: _____

Q6. Collaboration mode?

- ☐ Cooperative — shared goal, divide labor
- ☐ Competitive — agents bid for tasks
- ☐ Mixed — cooperate within, compete across
- ☐ Adversarial — detect or counter others

Which agents compete? Which cooperate? Why?

Q7. Agent environment observability?

- ☐ Fully Observable ☐ Partially Observable
- ☐ Static (env does not change) ☐ Dynamic (env changes constantly)

What state is hidden? What needs to be estimated?

Memory type needed:

- ☐ No memory ☐ Short-term (session)
- ☐ Long-term (DB) ☐ Shared (blackboard)

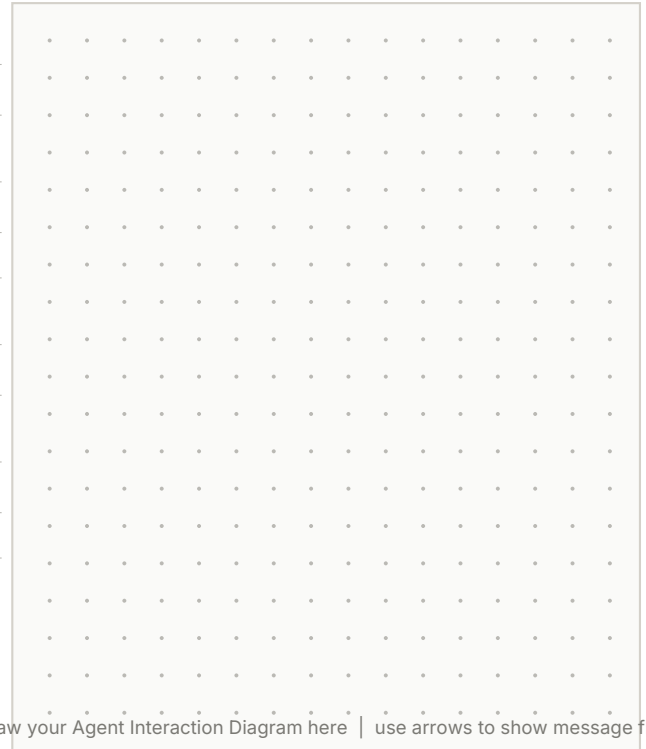
Q8. Frontend / Backend integration?

How does a user trigger agents? How do results come back?

API surface (REST / WebSocket / Event stream / other): _____

Agent Interaction Diagram

Draw agent nodes, message arrows, and hierarchy.



Draw your Agent Interaction Diagram here | use arrows to show message flow

Notes / Design Decisions:

My Key Insight:

ADK: ParallelAgent(anomaly + risk scoring) -> SequentialAgent(score -> action -> notify)

A2A: External credit bureau, fraud blacklist, and regulatory reporting system via A2A.

A city with 500+ intersections, 3 metro lines, and 50,000 parking spots needs unified real-time traffic management. Emergency vehicle response times average 9 minutes (target: 6). Morning rush-hour gridlock costs the economy \$1.2M/day in lost productivity. Current systems (traffic signals, transit, parking) are siloed and do not share data.

Success Metrics	Emergency vehicle response time (min) Average congestion index Parking utilization %
Key Constraints	Traffic agent cannot override manually set school zone times. Emergency routing is highest priority.

Q1. What specific problem are you solving? Write a clear 2-sentence problem statement.

Include: who is affected, what is failing, what is the measurable impact.

Q2. Why do you need a Multi-Agent System rather than a single agent?

Check all that apply:

- ☐ Problem too large for one context window
☐ Multiple independent parallel workstreams
- ☐ Requires distinct specialized expertise per domain
☐ Needs fault tolerance (one agent fails, others continue)
- ☐ Real-time coordination across physical systems/zones
☐ Different agents need different tools/permissions

Explain your reasoning:

Q3. How many agents do you need? List them below with their primary role.

Hint for this scenario: TrafficSignalAgent, EmergencyRoutingAgent, PublicTransportAgent, ParkingGuidanceAgent, IncidentResponseAgent

#	Agent Name	Type (LLM / Code / Both)	Primary Role (one sentence)
1			
2			
3			
4			
5			
6			

Q4. What does each agent do? Detail its functions and tools.

List agent's goal, tools it needs, what data it reads, and what it outputs.

Agent Name	Goal / Task	Tools Needed	Output / State Key

Q5. How will agents communicate?

Hint: MQTT sensor streams + REST APIs; FIPA REQUEST for signal changes; Draw agent nodes, message arrows, and hierarchy.
broadcast INFORM for incidents

Message Language:

Key Ontology Terms: _____

Protocol / Standard:

Sample Message:

Q6. Collaboration mode?

- ☐ Cooperative — shared goal, divide labor
- ☐ Competitive — agents bid for tasks
- ☐ Mixed — cooperate within, compete across
- ☐ Adversarial — detect or counter others

Which agents compete? Which cooperate? Why?

Q7. Agent environment observability?

- ☐ Fully Observable ☐ Partially Observable
 - ☐ Static (env does not change) ☐ Dynamic (env changes constantly)

What state is hidden? What needs to be estimated?

Memory type needed:

- ☐ No memory
 - ☐ Long-term (DB)
 - ☐ Short-term (session)
 - ☐ Shared (blackboard)

Q8. Frontend / Backend integration?

How does a user trigger agents? How do results come back?

API surface (REST / WebSocket / Event stream / other):

Agent Interaction Diagram

5. Draw agent nodes, message arrows, and hierarchy.

Draw your Agent Interaction Diagram here | use arrows to show message flow

Draw your Agent Interaction Diagram here | use arrows to show message flow

Notes / Design Decisions:

My Key Insight:

**ADK: LoopAgent for continuous signal optimization;
ParallelAgent for simultaneous zone management**

A2A: City platform connects traffic, transit authority, and parking via A2A. Cross-department data sharing.

A B2B SaaS company handles 8,000 support tickets per day. First-contact resolution rate is only 45% and average first response time is 4.2 hours. 30% of tickets are duplicates of known issues. Senior engineers spend 60% of their time on routine Tier-1 questions, leaving complex architectural issues unresolved for days.

Success Metrics	First-contact resolution % First response time (hrs) CSAT score
Key Constraints	Customer data cannot be shared between tenants. Escalation requires human engineer for Tier-3.

Q1. What specific problem are you solving? Write a clear 2-sentence problem statement.

Include: who is affected, what is failing, what is the measurable impact.

Q2. Why do you need a Multi-Agent System rather than a single agent?

Check all that apply:

- ☐ Problem too large for one context window
 ☐ Multiple independent parallel workstreams
- ☐ Requires distinct specialized expertise per domain
 ☐ Needs fault tolerance (one agent fails, others continue)
- ☐ Real-time coordination across physical systems/zones
 ☐ Different agents need different tools/permissions

Explain your reasoning:

Q3. How many agents do you need? List them below with their primary role.

Hint for this scenario: IntentClassifierAgent, KnowledgeBaseAgent, ResponseDraftAgent, EscalationAgent, FeedbackLearningAgent

#	Agent Name	Type (LLM / Code / Both)	Primary Role (one sentence)
1			
2			
3			
4			
5			
6			

Q4. What does each agent do? Detail its functions and tools.

List agent's goal, tools it needs, what data it reads, and what it outputs.

Agent Name	Goal / Task	Tools Needed	Output / State Key

Q5. How will agents communicate?

Hint: Natural language tickets + structured JSON metadata; FIPA INFORM for resolution status

Message Language: _____

Key Ontology Terms: _____

Protocol / Standard: _____

Sample Message: _____

Q6. Collaboration mode?

- ☐ Cooperative — shared goal, divide labor
- ☐ Competitive — agents bid for tasks
- ☐ Mixed — cooperate within, compete across
- ☐ Adversarial — detect or counter others

Which agents compete? Which cooperate? Why?

Q7. Agent environment observability?

- ☐ Fully Observable ☐ Partially Observable
- ☐ Static (env does not change) ☐ Dynamic (env changes constantly)

What state is hidden? What needs to be estimated?

Memory type needed:

- ☐ No memory ☐ Short-term (session)
- ☐ Long-term (DB) ☐ Shared (blackboard)

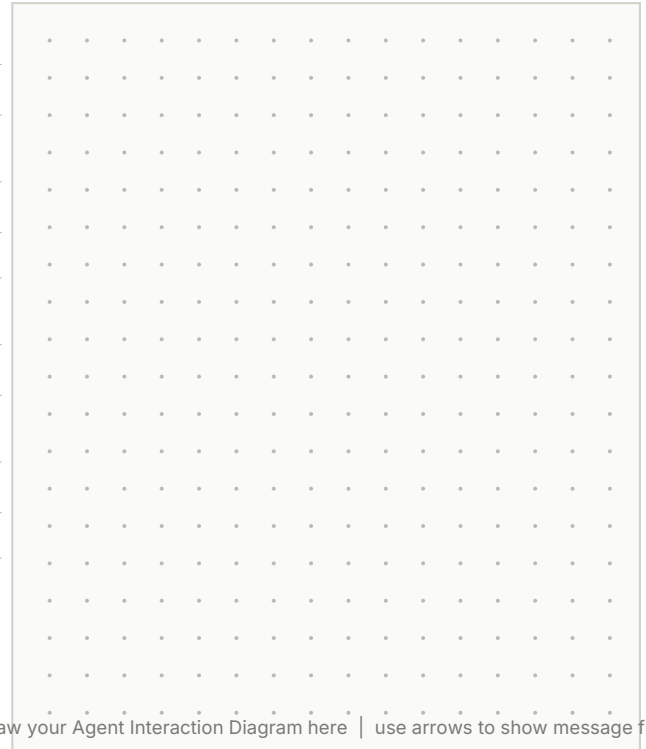
Q8. Frontend / Backend integration?

How does a user trigger agents? How do results come back?

API surface (REST / WebSocket / Event stream / other):

Agent Interaction Diagram

Draw agent nodes, message arrows, and hierarchy.



Draw your Agent Interaction Diagram here | use arrows to show message flow

Notes / Design Decisions:

My Key Insight:

ADK: Coordinator LlmAgent dispatches to specialists; LoopAgent for follow-up until resolved

A2A: Zendesk/Jira ticketing, Slack, and CRM connected via A2A for cross-tool visibility.

A global electronics manufacturer sources from 200 suppliers across 15 countries. Last year, 3 major disruptions (typhoon, factory fire, port strike) caused \$45M in losses. Average detection lag was 72 hours; re-routing negotiations took 2 weeks of manual work. The company needs real-time detection and automated response within 4 hours of any event.

Success Metrics	Disruption detection lag (hrs) Re-routing completion time (days) Supply risk score
Key Constraints	Cannot cancel contracts unilaterally. Re-routing proposals need procurement manager approval over \$100k.

Q1. What specific problem are you solving? Write a clear 2-sentence problem statement.

Include: who is affected, what is failing, what is the measurable impact.

Q2. Why do you need a Multi-Agent System rather than a single agent?

Check all that apply:

- ☐ Problem too large for one context window

☐ Multiple independent parallel workstreams
- ☐ Requires distinct specialized expertise per domain

☐ Needs fault tolerance (one agent fails, others continue)
- ☐ Real-time coordination across physical systems/zones

☐ Different agents need different tools/permissions

Explain your reasoning:

Q3. How many agents do you need? List them below with their primary role.

Hint for this scenario: RiskMonitorAgent, SupplierAuditAgent, LogisticsRerouteAgent, DemandForecastAgent, NegotiationAgent

#	Agent Name	Type (LLM / Code / Both)	Primary Role (one sentence)
1			
2			
3			
4			
5			
6			

Q4. What does each agent do? Detail its functions and tools.

List agent's goal, tools it needs, and what it outputs.

Agent Name	Goal / Task	Tools Needed	Output / State Key

Q5. How will agents communicate?

Hint: News/event feeds + EDI messages; FIPA CFP (Call-for-Proposal) for logistics re-routing bids

Message Language: _____

Key Ontology Terms: _____

Protocol / Standard: _____

Sample Message: _____

Q6. Collaboration mode?

- ☐ Cooperative — shared goal, divide labor
- ☐ Competitive — agents bid for tasks
- ☐ Mixed — cooperate within, compete across
- ☐ Adversarial — detect or counter others

Which agents compete? Which cooperate? Why?

Q7. Agent environment observability?

- ☐ Fully Observable ☐ Partially Observable
- ☐ Static (env does not change) ☐ Dynamic (env changes constantly)

What state is hidden? What needs to be estimated?

Memory type needed:

- ☐ No memory ☐ Short-term (session)
- ☐ Long-term (DB) ☐ Shared (blackboard)

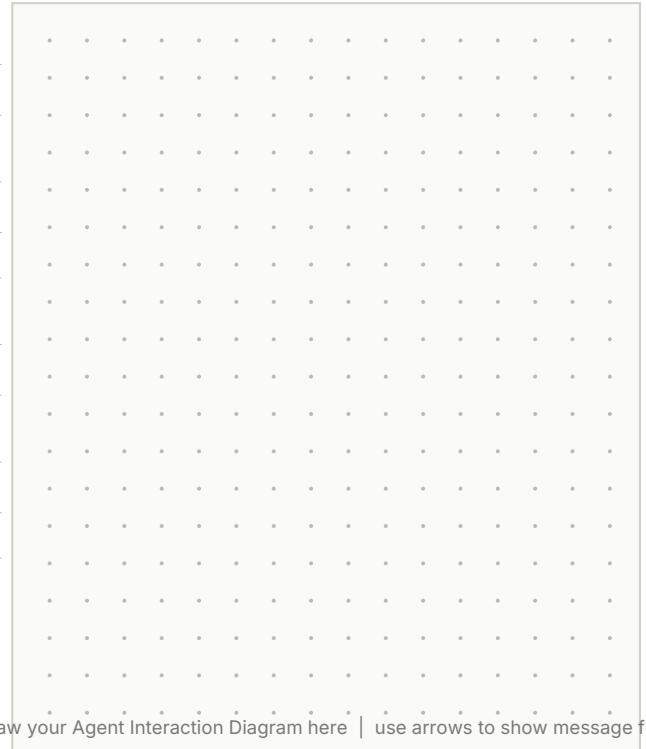
Q8. Frontend / Backend integration?

How does a user trigger agents? How do results come back?

API surface (REST / WebSocket / Event stream / other):

Agent Interaction Diagram

Draw agent nodes, message arrows, and hierarchy.



Draw your Agent Interaction Diagram here | use arrows to show message flow

Notes / Design Decisions:

My Key Insight:

ADK: LoopAgent(monitor risks) -> SequentialAgent(assess -> reroute -> notify). Human-in-loop for large contracts.

A2A: Supplier portals, 3PL logistics APIs, and ERP system connected via A2A.

A real estate investment fund with \$500M AUM needs to screen 500+ properties per month. Manual due diligence currently takes 3 weeks per property. The fund misses 30% of deals because analysis is too slow. Each deal requires parallel assessment across market conditions, legal compliance, financial modeling, and ESG (environmental/social/governance) criteria.

Success Metrics	Deals screened per month Due diligence time per deal (days) Investment decision accuracy %
Key Constraints	Legal opinions require licensed lawyer sign-off. ESG data from third parties may be unreliable.

Q1. What specific problem are you solving? Write a clear 2-sentence problem statement.

Include: who is affected, what is failing, what is the measurable impact.

Q2. Why do you need a Multi-Agent System rather than a single agent?

Check all that apply:

- ☐ Problem too large for one context window
 ☐ Multiple independent parallel workstreams
- ☐ Requires distinct specialized expertise per domain
 ☐ Needs fault tolerance (one agent fails, others continue)
- ☐ Real-time coordination across physical systems/zones
 ☐ Different agents need different tools/permissions

Explain your reasoning:

Q3. How many agents do you need? List them below with their primary role.

Hint for this scenario: MarketResearchAgent, PropertyValuationAgent, LegalDueDiligenceAgent, FinancialModelAgent, ESGScorerAgent

#	Agent Name	Type (LLM / Code / Both)	Primary Role (one sentence)
1			
2			
3			
4			
5			
6			

Q4. What does each agent do? Detail its functions and tools.

List agent's goal, tools it needs, what data it reads, and what it outputs.

Agent Name	Goal / Task	Tools Needed	Output / State Key

Q5. How will agents communicate?

Hint: Property data APIs + FIPA PROPOSE/ACCEPT-PROPOSAL for structured recommendation workflow

Message Language: _____

Key Ontology Terms: _____

Protocol / Standard: _____

Sample Message: _____

Q6. Collaboration mode?

- ☐ Cooperative — shared goal, divide labor
- ☐ Competitive — agents bid for tasks
- ☐ Mixed — cooperate within, compete across
- ☐ Adversarial — detect or counter others

Which agents compete? Which cooperate? Why?

Q7. Agent environment observability?

- ☐ Fully Observable ☐ Partially Observable
- ☐ Static (env does not change) ☐ Dynamic (env changes constantly)

What state is hidden? What needs to be estimated?

Memory type needed:

- ☐ No memory ☐ Short-term (session)
- ☐ Long-term (DB) ☐ Shared (blackboard)

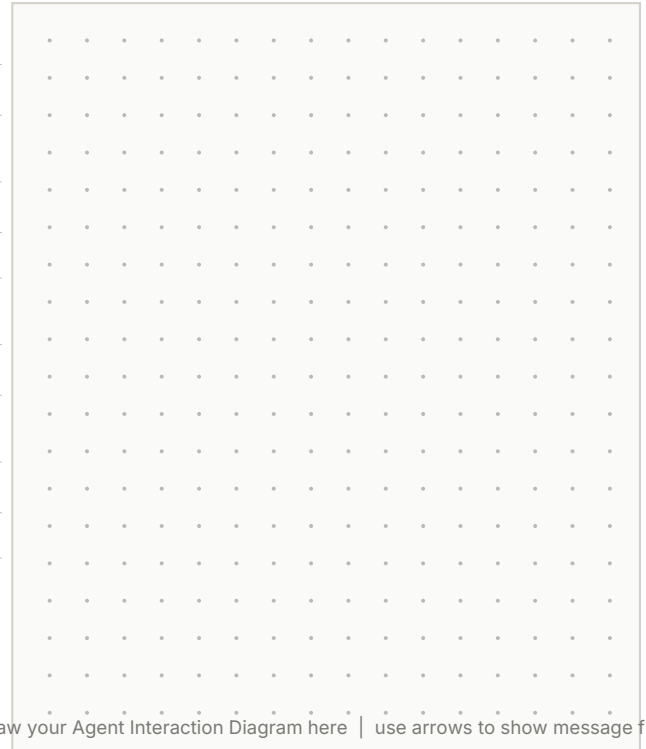
Q8. Frontend / Backend integration?

How does a user trigger agents? How do results come back?

API surface (REST / WebSocket / Event stream / other):

Agent Interaction Diagram

Draw agent nodes, message arrows, and hierarchy.



Draw your Agent Interaction Diagram here | use arrows to show message flow

Notes / Design Decisions:

My Key Insight:

ADK: ParallelAgent(market + legal + financial + ESG) -> SequentialAgent(consolidate -> recommend)

A2A: Land registry DB, international valuation APIs, legal research tools via A2A.

A restaurant chain with 200 locations across Thailand must comply with FDA food safety regulations. Last year, 3 foodborne illness incidents caused brand damage and \$2M in liability. Current temperature logs are manual and often falsified. Supplier audits happen only annually. Outbreak detection is reactive — the company learns of issues from customer complaints.

Success Metrics	Compliance score % Incident detection lag (hrs) Supplier audit completion rate %
Key Constraints	Regulatory reports must be submitted in FDA-specified XML format. Cannot close a branch without GM approval.

Q1. What specific problem are you solving? Write a clear 2-sentence problem statement.

Include: who is affected, what is failing, what is the measurable impact.

Q2. Why do you need a Multi-Agent System rather than a single agent?

Check all that apply:

- ☐ Problem too large for one context window
- ☐ Multiple independent parallel workstreams
- ☐ Requires distinct specialized expertise per domain
- ☐ Needs fault tolerance (one agent fails, others continue)
- ☐ Real-time coordination across physical systems/zones
- ☐ Different agents need different tools/permissions

Explain your reasoning:

Q3. How many agents do you need? List them below with their primary role.

Hint for this scenario: TempMonitorAgent, SupplierAuditAgent, InspectionSchedulerAgent, OutbreakDetectorAgent, RegulatoryReportAgent

#	Agent Name	Type (LLM / Code / Both)	Primary Role (one sentence)
1			
2			
3			
4			
5			
6			

Q4. What does each agent do? Detail its functions and tools.

List agent's goal, tools it needs, and what it outputs.

Agent Name	Goal / Task	Tools Needed	Output / State Key

Q5. How will agents communicate?

Hint: IoT sensor data (MQTT) + supplier audit JSON + HL7-like regulatory XML messages

Message Language:

Key Ontology Terms:

Protocol / Standard:

Sample Message:

Q6. Collaboration mode?

- ☐ Cooperative — shared goal, divide labor
- ☐ Competitive — agents bid for tasks
- ☐ Mixed — cooperate within, compete across
- ☐ Adversarial — detect or counter others

Which agents compete? Which cooperate? Why?

Q7. Agent environment observability?

- ☐ Fully Observable ☐ Partially Observable
- ☐ Static (env does not change) ☐ Dynamic (env changes constantly)

What state is hidden? What needs to be estimated?

Memory type needed:

- ☐ No memory ☐ Short-term (session)
- ☐ Long-term (DB) ☐ Shared (blackboard)

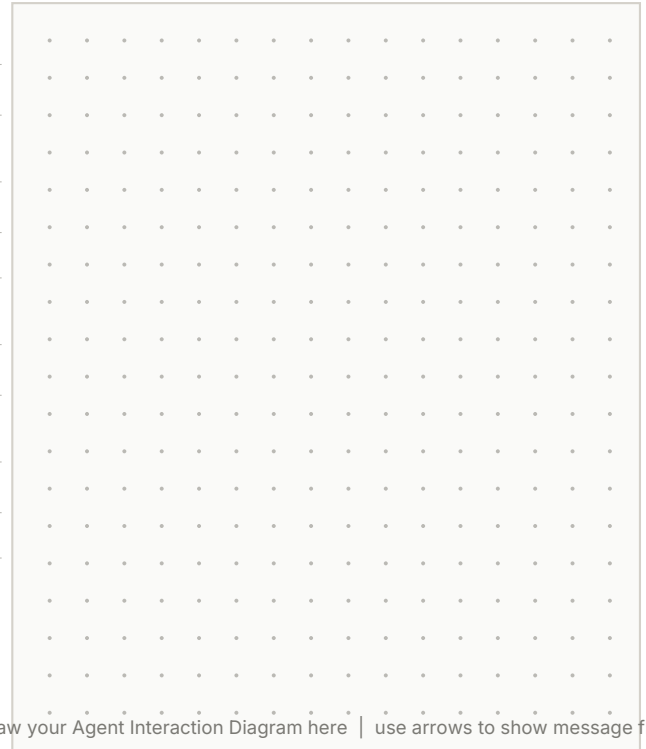
Q8. Frontend / Backend integration?

How does a user trigger agents? How do results come back?

API surface (REST / WebSocket / Event stream / other):

Agent Interaction Diagram

Draw agent nodes, message arrows, and hierarchy.



Draw your Agent Interaction Diagram here | use arrows to show message flow

Notes / Design Decisions:

My Key Insight:

ADK: LoopAgent(continuous temp monitoring) -> triggered SequentialAgent(detect -> alert -> report)

A2A: FDA reporting portal, supplier management system, and health dept system via A2A.

Workshop Reflection & Summary

You have designed 10 Multi-Agent Systems from scratch. Before you implement any of them, take 10 minutes to reflect on what you learned.

Which scenario surprised you most? Why?

Which scenario would you most want to implement? Which ADK pattern would you use?

What was the hardest design question across all scenarios?

- ☐ Q3: Choosing the right number of agents
- ☐ Q4: Defining each agent's tools precisely
- ☐ Q5: Designing the communication protocol
- ☐ Q6: Deciding cooperation vs competition
- ☐ Q7: Handling partial observability
- ☐ Q8: Fitting MAS into a real product architecture

What is the most important lesson you take away from this workshop?

Quick Decision Guide

Situation	Design Decision
Task requires sequential steps	Use SequentialAgent in ADK
Multiple independent data sources	Use ParallelAgent; merge via shared state
Continuous monitoring needed	Use LoopAgent with max_iterations
Dynamic task routing by intent	Use LlmAgent + transfer_to_agent()
High-stakes action needed	Add Human-in-the-Loop via FunctionTool
Cross-framework agents	Use Google A2A protocol
Agents need to bid for tasks	Use FIPA Contract Net Protocol (CFP)
Agents share domain knowledge	Define ontology first; use INFORM performative
Environment is partially observable	Add state estimation agent or Bayesian model
Frontend needs live updates	Use WebSocket or event streaming from orchestrator

Next step: Pick your favourite scenario, open the MAS Coding Exercises PDF, and implement it. Your design from this worksheet is your architecture document.